Assignment 1: Air-Quality Data Cleaning Using R

Topics: Loops, functions, error handling, and advanced missing data handling (`NA`, `NULL`, `NaN`)

Task 1 — Import and inspect the dataset

```
data_path <- "/content/PRSA_Data_Aotizhongxin_20130301-20170228.csv"

air_data <- tryCatch({
  if (!file.exists(data_path)) {
    stop("File not found. Upload the station CSV and update data_path.")
  }
  imported <- read.csv(
    data_path,
    stringsAsFactors = FALSE,
    na.strings = c("", "NA", "NaN"),
    check.names = FALSE
  )
  if (!is.data.frame(imported) || ncol(imported) == 0) {
    stop("The selected file does not appear to be a valid CSV data table.")
  }
  imported
}, error = function(e) {
  message("Import error: ", conditionMessage(e))
  NULL
})

if (is.null(air_data)) {
  stop("Import failed. Upload the correct CSV and re-run this cell.")
}

cat("First six records:\\n")
print(head(air_data))
cat("\\nDataset structure:\\n")
str(air_data)
cat("\\nRows:", nrow(air_data), "\\n")
cat("Columns:", ncol(air_data), "\\n")
cat("Total missing values:", sum(is.na(air_data)), "\\n")
cat("Contains missing values:", anyNA(air_data), "\\n")
```

```
First six records:\n  No year month day hour PM2.5 PM10 SO2 NO2  CO O3 TEMP   PRES  DEWP RAIN  wd
1  1 2013     3   1    0     4    4   4   7 300 77 -0.7 1023.0 -18.8    0 NNW
2  2 2013     3   1    1     8    8   4   7 300 77 -1.1 1023.2 -18.2    0   N
3  3 2013     3   1    2     7    7   5  10 300 73 -1.1 1023.5 -18.2    0 NNW
4  4 2013     3   1    3     6    6  11  11 300 72 -1.4 1024.5 -19.4    0  NW
5  5 2013     3   1    4     3    3  12  12 300 72 -2.0 1025.2 -19.5    0   N
6  6 2013     3   1    5     5    5  18  18 400 66 -2.2 1025.6 -19.6    0   N
  WSPM      station
1  4.4 Aotizhongxin
2  4.7 Aotizhongxin
3  5.6 Aotizhongxin
4  3.1 Aotizhongxin
5  2.0 Aotizhongxin
6  3.7 Aotizhongxin
\nDataset structure:\n'data.frame':     35064 obs. of  18 variables:
 $ No     : int  1 2 3 4 5 6 7 8 9 10 ...
 $ year   : int  2013 2013 2013 2013 2013 2013 2013 2013 2013 2013 ...
 $ month  : int  3 3 3 3 3 3 3 3 3 3 ...
 $ day    : int  1 1 1 1 1 1 1 1 1 1 ...
 $ hour   : int  0 1 2 3 4 5 6 7 8 9 ...
 $ PM2.5  : num  4 8 7 6 3 5 3 3 3 3 ...
 $ PM10   : num  4 8 7 6 3 5 3 6 6 8 ...
 $ SO2    : num  4 4 5 11 12 18 18 19 16 12 ...
 $ NO2    : num  7 7 10 11 12 18 32 41 43 28 ...
 $ CO     : int  300 300 300 300 300 400 500 500 500 400 ...
 $ O3     : num  77 77 73 72 72 66 50 43 45 59 ...
 $ TEMP   : num  -0.7 -1.1 -1.1 -1.4 -2 -2.2 -2.6 -1.6 0.1 1.2 ...
 $ PRES   : num  1023 1023 1024 1024 1025 ...
 $ DEWP   : num  -18.8 -18.2 -18.2 -19.4 -19.5 -19.6 -19.1 -19.1 -19.2 -19.3 ...
 $ RAIN   : num  0 0 0 0 0 0 0 0 0 0 ...
 $ wd     : chr  "NNW" "N" "NNW" "NW" ...
 $ WSPM   : num  4.4 4.7 5.6 3.1 2 3.7 2.5 3.8 4.1 2.6 ...
 $ station: chr  "Aotizhongxin" "Aotizhongxin" "Aotizhongxin" "Aotizhongxin" ...
\nRows: 35064 \nColumns: 18 \nTotal missing values: 7271 \nContains missing values: TRUE \n
```

Task 2 — Understand `NA`, `NULL`, and `NaN`

```
temperature_example <- c(28, 30, NA, 32)
missing_object <- NULL
undefined_value <- 0 / 0

cat("temperature_example:\\n")
print(temperature_example)
cat("is.na(temperature_example):\\n")
print(is.na(temperature_example))

cat("\\nmissing_object:\\n")
print(missing_object)
cat("is.null(missing_object):\\n")
print(is.null(missing_object))

cat("\\nundefined_value:\\n")
print(undefined_value)
cat("is.nan(undefined_value):\\n")
print(is.nan(undefined_value))
cat("is.na(undefined_value):\\n")
print(is.na(undefined_value))

# In R, is.na(NaN) is TRUE too; is.nan() specifically identifies NaN.
```

```
temperature_example:\n[1] 28 30 NA 32
is.na(temperature_example):\n[1] FALSE FALSE  TRUE FALSE
\nmissing_object:\nNULL
is.null(missing_object):\n[1] TRUE
\nundefined_value:\n[1] NaN
is.nan(undefined_value):\n[1] TRUE
is.na(undefined_value):\n[1] TRUE
```

Task 3 — Create a missing-value summary function

```
missing_summary <- function(data, variables = c(
  "PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM", "wd"
)) {
  if (!is.data.frame(data)) stop("Input must be a data frame.")

  absent <- setdiff(variables, names(data))
  if (length(absent) > 0) {
    warning("These requested variables are absent: ",
            paste(absent, collapse = ", "))
  }

  present <- intersect(variables, names(data))
  if (length(present) == 0) {
    return(data.frame(
      Variable = character(), Total_Records = integer(),
      Missing_Values = integer(), Missing_Percentage = numeric()
    ))
  }

  result <- do.call(rbind, lapply(present, function(variable_name) {
    missing_count <- sum(is.na(data[[variable_name]]))
    total_count <- nrow(data)
    percentage <- if (total_count == 0) 0 else 100 * missing_count / total_count

    if (percentage > 20) {
      warning(sprintf("%s has %.2f%% missing values (>20%%).",
                      variable_name, percentage), call. = FALSE)
    }

    data.frame(
      Variable = variable_name,
      Total_Records = total_count,
      Missing_Values = missing_count,
      Missing_Percentage = round(percentage, 2),
      stringsAsFactors = FALSE
    )
  }))
  rownames(result) <- NULL
  result
}

missing_summary_before <- missing_summary(air_data)
print(missing_summary_before)
```

```
  Variable Total_Records Missing_Values Missing_Percentage
1    PM2.5         35064            925              2.64
2     PM10         35064            718              2.05
3      SO2         35064            935              2.67
4      NO2         35064           1023              2.92
5     TEMP         35064             20              0.06
6     WSPM         35064             14              0.04
7       wd         35064             81              0.23
```

Task 4 — Identify invalid numerical results in `pollution_ratio`

```
required_ratio_columns <- c("PM2.5", "PM10")
missing_ratio_columns <- setdiff(required_ratio_columns, names(air_data))
if (length(missing_ratio_columns) > 0) {
  stop("Cannot calculate pollution_ratio. Missing columns: ",
       paste(missing_ratio_columns, collapse = ", "))
}

air_data[["PM2.5"]] <- suppressWarnings(as.numeric(air_data[["PM2.5"]]))
air_data[["PM10"]] <- suppressWarnings(as.numeric(air_data[["PM10"]]))
air_data$pollution_ratio <- air_data[["PM2.5"]] / air_data[["PM10"]]

cat("NA count (includes NaN in R):", sum(is.na(air_data$pollution_ratio)), "\\n")
cat("NaN count:", sum(is.nan(air_data$pollution_ratio)), "\\n")
cat("Positive infinity count:",
    sum(is.infinite(air_data$pollution_ratio) & air_data$pollution_ratio > 0, na.rm = TRUE), "\\n")
cat("Negative infinity count:",
    sum(is.infinite(air_data$pollution_ratio) & air_data$pollution_ratio < 0, na.rm = TRUE), "\\n")

invalid_ratio <- is.nan(air_data$pollution_ratio) | is.infinite(air_data$pollution_ratio)
air_data$pollution_ratio[invalid_ratio] <- NA_real_
cat("Invalid values replaced with NA:", sum(invalid_ratio), "\\n")
cat("Remaining NaN:", sum(is.nan(air_data$pollution_ratio)), "\\n")
cat("Remaining infinite values:", sum(is.infinite(air_data$pollution_ratio)), "\\n")
```

```
NA count (includes NaN in R): 934 \nNaN count: 0 \nPositive infinity count: 0 \nNegative infinity count: 0 \nInvalid values
```

Task 5 — Handle missing numerical values using a loop

```
numeric_variables <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM")

# Capture original missing counts before any imputation.
selected_variables <- c(numeric_variables, "wd")
missing_before <- vapply(selected_variables, function(v) {
  if (v %in% names(air_data)) sum(is.na(air_data[[v]])) else NA_integer_
}, integer(1))

numeric_imputation_log <- data.frame(
  Variable = character(), Missing_Before = integer(),
  Median_Used = numeric(), Missing_After = integer(),
  stringsAsFactors = FALSE
)

for (variable_name in numeric_variables) {
  if (!variable_name %in% names(air_data)) {
    message("Skipping ", variable_name, ": column does not exist.")
    next
  }

  original_values <- air_data[[variable_name]]
  numeric_values <- suppressWarnings(as.numeric(original_values))
  newly_invalid <- sum(is.na(numeric_values) & !is.na(original_values))
  if (newly_invalid > 0) {
    message(variable_name, ": ", newly_invalid,
            " non-numeric value(s) converted to NA.")
  }
  air_data[[variable_name]] <- numeric_values

  missing_count_before <- sum(is.na(air_data[[variable_name]]))
  median_value <- if (all(is.na(air_data[[variable_name]]))) {
    NA_real_
  } else {
    median(air_data[[variable_name]], na.rm = TRUE)
  }

  if (is.na(median_value) || !is.finite(median_value)) {
    message("Cannot impute ", variable_name,
```

```
            ": no finite median is available. Missing values remain unchanged.")
    missing_count_after <- sum(is.na(air_data[[variable_name]]))
  } else {
    air_data[[variable_name]][is.na(air_data[[variable_name]])] <- median_value
    missing_count_after <- sum(is.na(air_data[[variable_name]]))
  }

  numeric_imputation_log <- rbind(numeric_imputation_log, data.frame(
    Variable = variable_name,
    Missing_Before = missing_count_before,
    Median_Used = median_value,
    Missing_After = missing_count_after,
    stringsAsFactors = FALSE
  ))

  cat("\\nVariable:", variable_name,
      "\\nMissing before:", missing_count_before,
      "\\nMedian used:", ifelse(is.na(median_value), "Not available", median_value),
      "\\nMissing after:", missing_count_after, "\\n")
}
print(numeric_imputation_log)
```

```
\nVariable: PM2.5 \nMissing before: 925 \nMedian used: 58 \nMissing after: 0 \n\nVariable: PM10 \nMissing before: 718 \nMedi
1    PM2.5            925        58.0             0
2     PM10            718        87.0             0
3      SO2            935         9.0             0
4      NO2           1023        53.0             0
5     TEMP             20        14.5             0
6     WSPM             14         1.4             0
```

Task 6 — Handle missing categorical values in `wd`

```
calculate_mode <- function(x) {
  observed <- x[!is.na(x)]
  if (length(observed) == 0) return(NA)
  frequencies <- table(observed)
  names(frequencies)[which.max(frequencies)]
}

if (!"wd" %in% names(air_data)) {
  message("Cannot impute wd: column does not exist.")
} else {
  wd_missing_before <- sum(is.na(air_data$wd))
  wd_mode <- calculate_mode(air_data$wd)

  if (is.na(wd_mode)) {
    message("Cannot impute wd: all values are missing.")
  } else {
    air_data$wd[is.na(air_data$wd)] <- wd_mode
  }

  wd_missing_after <- sum(is.na(air_data$wd))
  cat("Mode of wd:", ifelse(is.na(wd_mode), "Not available", wd_mode), "\\n")
  cat("Missing before:", wd_missing_before, "\\n")
  cat("Missing after:", wd_missing_after, "\\n")
}
```

```
Mode of wd: NE \nMissing before: 81 \nMissing after: 0 \n
```

Task 7 — Implement error handling with `clean_variable()`

```
clean_variable <- function(data, variable_name) {
  tryCatch({
    if (!is.data.frame(data)) stop("Input must be a data frame.")
    if (length(variable_name) != 1 || !variable_name %in% names(data)) {
      stop("Variable '", paste(variable_name, collapse = ", "),
           "' does not exist in the dataset.")
    }

    variable <- data[[variable_name]]
    if (!is.numeric(variable)) {
      stop("Variable '", variable_name, "' is not numerical.")
    }
    if (all(is.na(variable))) {
      stop("Variable '", variable_name, "' contains only missing values.")
    }

    median_value <- median(variable, na.rm = TRUE)
```

```
    if (length(median_value) != 1 || is.na(median_value) ||
        !is.finite(median_value)) {
      stop("Median could not be calculated for '", variable_name, "'.")
    }

    variable[is.na(variable)] <- median_value
    message("Successfully cleaned variable: ", variable_name)
    variable
  }, error = function(e) {
    message("Could not clean variable: ", conditionMessage(e))
    NULL
  })
}

cleaned_temp <- clean_variable(air_data, "TEMP")
if (!is.null(cleaned_temp)) air_data$TEMP <- cleaned_temp

# Demonstrate error cases; these messages should not terminate the notebook.
invisible(clean_variable(air_data, "COLUMN_DOES_NOT_EXIST"))
if ("wd" %in% names(air_data)) invisible(clean_variable(air_data, "wd"))
invisible(clean_variable(data.frame(all_missing = c(NA_real_, NA_real_)),
                         "all_missing"))
```

```
Successfully cleaned variable: TEMP

Could not clean variable: Variable 'COLUMN_DOES_NOT_EXIST' does not exist in the dataset.

Could not clean variable: Variable 'wd' is not numerical.

Could not clean variable: Variable 'all_missing' contains only missing values.
```

Task 8 — Compare missing values before and after cleaning

```
selected_variables <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM", "wd")

comparison_table <- data.frame(
  Variable = selected_variables,
  Missing_Before = vapply(selected_variables, function(v) {
    if (v %in% names(missing_before)) as.integer(missing_before[[v]]) else NA_integer_
  }, integer(1)),
  Missing_After = vapply(selected_variables, function(v) {
    if (v %in% names(air_data)) as.integer(sum(is.na(air_data[[v]]))) else NA_integer_
  }, integer(1)),
  stringsAsFactors = FALSE
)
comparison_table$Values_Replaced <- comparison_table$Missing_Before -
  comparison_table$Missing_After

print(comparison_table)

if (all(!is.na(comparison_table$Missing_After)) &&
    all(comparison_table$Missing_After == 0)) {
  cat("\\nAll selected variables have zero missing values after cleaning.\\n")
} else {
  cat("\\nSome selected variables still contain missing values or are absent. ",
      "Check Missing_After values and absent columns.\\n", sep = "")
}
```

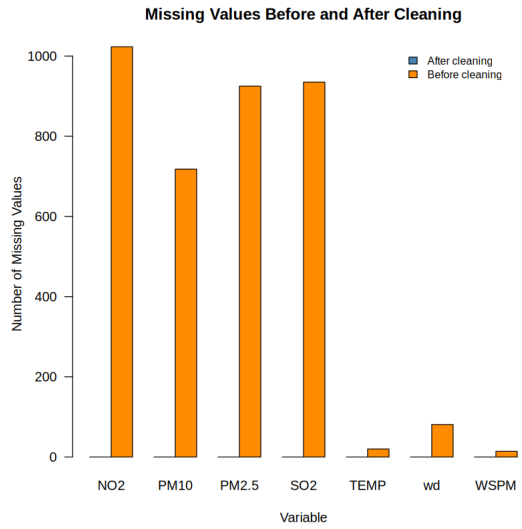
```
     Variable Missing_Before Missing_After Values_Replaced
PM2.5    PM2.5            925             0              925
PM10      PM10            718             0              718
SO2        SO2            935             0              935
NO2        NO2           1023             0             1023
TEMP      TEMP             20             0               20
WSPM      WSPM             14             0               14
wd          wd             81             0               81
\nAll selected variables have zero missing values after cleaning.\n
```

Task 9 — Visualize missing values before and after cleaning

```
plot_data <- rbind(
  data.frame(Variable = comparison_table$Variable,
             Stage = "Before cleaning",
             Missing = comparison_table$Missing_Before),
  data.frame(Variable = comparison_table$Variable,
             Stage = "After cleaning",
             Missing = comparison_table$Missing_After)
)
```

```
plot_data <- plot_data[complete.cases(plot_data), ]
plot_matrix <- xtabs(Missing ~ Variable + Stage, data = plot_data)

barplot(
  t(plot_matrix),
  beside = TRUE,
  col = c("steelblue", "darkorange"),
  main = "Missing Values Before and After Cleaning",
  xlab = "Variable",
  ylab = "Number of Missing Values",
  legend.text = TRUE,
  args.legend = list(x = "topright", bty = "n", cex = 0.8),
  las = 1
)
```



Task 10 — Export the cleaned dataset

```
output_file <- "/content/cleaned_air_quality_data.csv"
write.csv(air_data, file = output_file, row.names = FALSE, na = "")
cat("Cleaned dataset exported to: ", output_file, "\\n")
cat("Export successful: ", file.exists(output_file), "\\n")

# In Colab, uncomment the next lines to download the CSV to your computer:
# install.packages("googledrive") # not required for download helper
# In a Python-backed Colab runtime, use the Python cell below instead.
```

```
Cleaned dataset exported to:  /content/cleaned_air_quality_data.csv \nExport successful:  TRUE \n
```

The Beijing Multi-Site Air Quality dataset contains hourly pollutant and weather measurements, and some records have missing observations. In this practical, the dataset was imported and inspected to determine its structure and missing-value count. The difference between `NA`, `NULL`, and `NaN` was demonstrated using R's built-in identification functions. A reusable `missing_summary()` function reported missing counts and percentages for selected variables and warned when missingness exceeded 20%. The `pollution_ratio` variable was checked for undefined and infinite values, which were converted to `NA`. A loop applied median imputation to numerical variables, while a mode-based function handled missing wind-direction values. The reusable `clean_variable()` function demonstrated error handling using `tryCatch()`. Before-and-after tables and a bar chart assessed the cleaning results, and the processed dataset was exported to CSV without row numbers. Review the generated tables to confirm the actual results.